CS 412: Spring '26
Introduction To Data Mining

Assignment 4

(Due Wednesday, April 15, 23:59)

- The homework is due on Wednesday, April 15, 23:59
- Please write your code entirely by yourself. All programming needs to be in Python 3.
- Do NOT add any third-party libraries in your code. Built-in Python libraries are allowed.
- You are provided two sample test cases on Coursera, you can try debugging your code with minsup values of 2 with the given sample inputs.
- Please submit your code on Gradescope.
- Late submission policy: there will be a 24-hour grace period without any grade reduction.

Part 1: Apriori Algorithm

Problem Description: In Part 1, you will implement a frequent itemset mining algorithm based on **Apriori method with pruning** with constraints. Given a transaction database TDB, prices database and a minimum support threshold *minsup*, your algorithm should simulate the Aprirori method with pruning - returning all the candidate itemsets and the frequent itemsets at each algorithm scan based on the constraints. Only the following two constraints will be applied,

$$\text{sum(S.price)} \leq k$$
$$\text{min(S.price)} \geq m,$$

where sum() corresponds to the sum of all prices of the items in the sequence and min() corresponds to the minimum price of all items in the sequence. k and m will be provided as inputs. We will test your code on relatively small transaction databases (maximum 15 transactions of length 10). Please make sure the runtime of your code does not exceed 10 seconds for such small databases. *You will not get any credit if your code does not work.*

Input Format: The input will be a plain text file with a transaction database, with each line corresponding to a transaction composed of a string of letters. Each letter in a transaction corresponds to an item. For example, the transaction database *Test-1.txt* is as in the following Table 1. The costs file will also be a text file with each line an item and its corresponding cost (separated by a comma) as shown in Table 2.

ACD
BCE
$ABCE$
BE

Table 1: Transactions

A,7
B,3
C,10
D,6

Table 2: Item costs

Output Format: Your code will implement a function called apriori based on the Apriori algorithm with pruning. It will return a 3-level nested dictionary. Figure 1 shows the simulation of the Apriori algorithm **(without constraints)** with pruning for an example. The expected output (3-level nested dictionary to be returned from the apriori function of your code) is shown in Figure 2. Consider the 3 levels of the dictionary as *outer*, *middle*, and *inner* levels.

- The keys of the outer level will denote the scans (or iterations) of the algorithm. For example, in Figure 1, the algorithm terminates after 3 scans and so in the dictionary of Figure 2, we have 3 elements in the *outer* dictionary, where the keys of these 3 elements are integers 1, 2, and 3 denoting the first, second and third scans of the algorithm, respectively. The scan numbers must start from 1 and should of integer data type.
- The value of each scan no.(i.e., each key in the *outer* layer) is a dictionary, which are the *middle* layer dictionaries. In Figure 1, the algorithm generates the candidate itemsets and the frequent itemsets in each scan. So each *middle* dictionary will have two elements: the key c denoting the candidate itemsets and the key f denoting the frequent itemsets. The data type of keys c and f should be string.
- The value for the keys c and f will be dictionaries denoting the candidate itemsets and the frequent itemsets of the corresponding scan. The keys of these dictionaries will be of string data type denoting the itemsets. The values will be of integer data type denoting the support of the associated itemset.

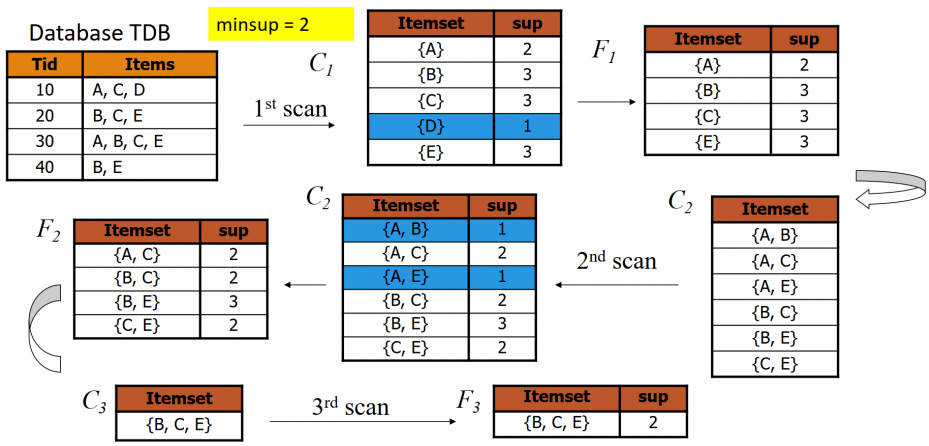

Figure 1: Simulation of *Test-1.txt*

Implementation details: The following notes further specify how your algorithm should behave.

- Pruning: While creating the candidate itemsets at every scan, you must apply pruning. For example, in Figure 1, at the 2nd scan, merging AC and BC can generate the candidate ABC for the 3rd scan, but as a subset AB of ABC is absent in the frequent set F_2, ABC is pruned and not included in the candidate set C_3. Similarly, the ABC is absent in the corresponding *inner* dictionary of Figure 2.
- Sorting: The alphabets in the strings of the keys of the *inner* dictionaries should be alphabetically sorted. For example, BCE should not be any of BEC, CBE, CEB, ECB, EBC.
- Terminating: If the frequent itemsets of a scan has only one itemset, the algorithm will terminate and no further scan will be done. For example, in Figure 1, F_3 has only one itemset BCE, so the 4th scan was not performed. Also, if the candidate itemsets of a scan is empty, that scan will be discarded and won't be included in the output. For example, let's assume for some input, the frequent itemsets F_2 obtained at 2nd scan are AC, BC. So the candidate itemsets C_3 for the 3rd scan will be empty (ABC won't be in C_3 as AB is absent in F_2 and so ABC will be pruned). In this case, the output will not include the 3rd scan as both C_3 and F_3 are empty.
- Errors: If you get an error from the autograder that says the code could not be executed properly and suggests contacting the course staff, please first check carefully if your code is running into an infinite loop. An infinite loop is the most likely cause of this error.

What you have to submit: You need to submit a Python file named homework4.py. An incorrect filename will result in a Gradescope error. A starter code is posted on Coursera. You can add as many functions in your code as you need. Your code should be implemented in Python 3. Do NOT add any third-party packages in your code; you can use Python's built-in packages. Your code must include a function named apriori which takes the following five inputs:

- Path to a plain text file pointing to the transaction database.
- Path to a plain text file pointing to the item cost table.
- An integer (s), the minimum support.
- An integer (k), constraint on the sum of item prices.
- An integer (m), constraint on the minimum of item prices.

A call to the function will be like: `apriori(transaction_filename = "hw4_sample_input_1.txt", cost_filename = "hw4_sample_input_1_price.txt", s = 2, k = 20, m = 5)`

Outer Dictionary

Middle Dictionary

Inner Dictionary

```
{
    1: {
        "c": {
            "A": 2,
            "C": 3,
            "D": 1,
            "B": 3,
            "E": 3
        },
        "f": {
            "A": 2,
            "C": 3,
            "B": 3,
            "E": 3
        }
    },
    2: {
        "c": {
            "AC": 2,
            "AB": 1,
            "AE": 1,
            "BC": 2,
            "CE": 2,
            "BE": 3
        },
        "f": {
            "AC": 2,
            "BC": 2,
            "CE": 2,
            "BE": 3
        }
    },
    3: {
        "c": {
            "BCE": 2
        },
        "f": {
            "BCE": 2
        }
    }
}
```

datatype: string

datatype: integer

Figure 2: Expected output for *Test-1.txt*

Part 2: Mining Association Rules

Problem description: In Part 2, you will generate association rules from the frequent itemsets you mined in Part 1 and compute pattern evaluation measures. The general process is: (1) mine frequent itemsets, (2) from each frequent itemset Z, enumerate rules $X \Rightarrow Y$ where $X \cup Y = Z$, $X \cap Y = \varnothing$, $X \neq \varnothing$, $Y \neq \varnothing$, (3) compute metrics (4) filter by minimum confidence.

You will make use of the following rule quality measures. For a rule $X \Rightarrow Y$, where N is the total number of transactions in the input file (no filtering):

$$\text{supp}(X) = \frac{\#\{\text{transactions containing } X\}}{N}, \quad \text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}, \quad \text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{supp}(Y)},$$

$$\text{Jaccard}(X \Rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X) + \text{supp}(Y) - \text{supp}(X \cup Y)}.$$

Input format: The inputs are the same files and thresholds as Part 1 (`transaction_filename`, `price_filename`, `s`, `k`, `m`), plus a minimum confidence threshold `min_conf` in $[0, 1]$.

Output format: Your code will implement a function called mine_rules. It will return a dictionary. Only include rules with `confidence` $\geq$ `min_conf`. If no rules satisfy `min_conf`, return an empty dictionary $\{\}$. Each key in the dictionary is a rule written as $X \Rightarrow Y$ where both X and Y are strings of letters in *alphabetical* order (e.g., `"AB->C"`, `"A->BC"`). Each value is a dictionary with the following fields:

- `support_count`: integer count of transactions containing $X \cup Y$,
- `support`: support_count/N (rounded to 6 decimals),
- `confidence`: $\text{supp}(X \cup Y)/\text{supp}(X)$ (rounded to 6 decimals),
- `lift`: confidence/$\text{supp}(Y)$ (rounded to 6 decimals),
- `jaccard`: $\text{supp}(X \cup Y)/(\text{supp}(X) + \text{supp}(Y) - \text{supp}(X \cup Y))$ (rounded to 6 decimals).

Implementation details: The following notes further specify how your algorithm should behave.

- On both sides of the rule, letters must be alphabetically sorted.
- There must be **no spaces** in keys; use the exact `"X->Y"` format.
- Round `support`, `confidence`, `lift` , `jaccard` to 6 places using Pythons `round(x, 6)` before returning.
- **Hint:** First call your Part 1 `apriori` to obtain frequent itemsets and their supports. Build a support dictionary for *all* frequent sets across scans. For each frequent itemset Z with $|Z| \geq 2$, enumerate all non-empty proper subsets $X \subset Z$; put $Y = Z \setminus X$. Keep the rule only if X, Y are frequent (under s, k, m) and $\text{conf}(X \Rightarrow Y) \geq$ `min_conf`. Compute `support_count`, `support`, `confidence`, `lift` as defined above.

What you have to submit: Modify the Python file named homework4.py. Your code should be implemented in Python 3. Do NOT add any third-party packages in your code; you can use Python's built-in packages. Your code must include a function named mine_rules which takes the following five inputs:

- Path to a plain text file pointing to the transaction database.
- Path to a plain text file pointing to the item cost table.
- An integer (s), the minimum support.

- An integer (k), constraint on the sum of item prices.
- An integer (m), constraint on the minimum of item prices.
- A float (min_conf), the minimum confidence threshold.

A call to the function will be like: `mine_rules(transaction_filename = "hw4_sample_input_1.txt", cost_filename = "hw4_sample_input_1_price.txt", s = 2, k = 20, m = 5, min_conf = 0.6)`

Additional Guidelines

The assignment needs you to both understand algorithms for frequent itemset mining, in particular Apriori with pruning, as well as being able to implement the algorithm in Python. Here are some guidelines to consider for the homework:

- Please start early. It is less likely you will be able to do a satisfactory job if you start late.
- It is a good idea to make early progress on the assignment, so you can assess how long it will take: (a) start working on the assignment as soon as it is posted. Within the first week, you should have a sense of the parts that will be easier and parts that will need extra effort from you; (b) Solve an example (partly) by hand as a warm-up to get comfortable with the steps that you will have to code. For the warm-up, you can use the sample test cases provided on Coursera named hw4_sample_input_1.txt and hw4_sample_input_2.txt.
- Sample test cases have been given. The output files contain the final pruned set of patterns. We used $s = 2$ and $k = 20$ and $m = 5$ for the two test cases. Note: We will test the code with a different set of support thresholds and constraint conditions. An additional output file for Part 2 containing mine rules for the sample files has also been provided to you.